1. clear architecture for **dev / test / prod** clusters (node counts, network, HA design)

2. **diagrams** (text diagrams — no Mermaid) for each environment showing master/worker components and ingress/load balancer placement

3. an opinionated, reproducible **kubeadm + Ansible** approach to create clusters (dev = single control plane, prod = HA control-plane) — with copy-pasteable playbook snippets and kubeadm config examples

4. Kubernetes **manifests** to deploy a simple webapp (nginx-based) + Service + Ingress and instructions to install the NGINX Ingress Controller (Helm) and how to access it

5. operational notes (storage, backups, monitoring, security, DNS)

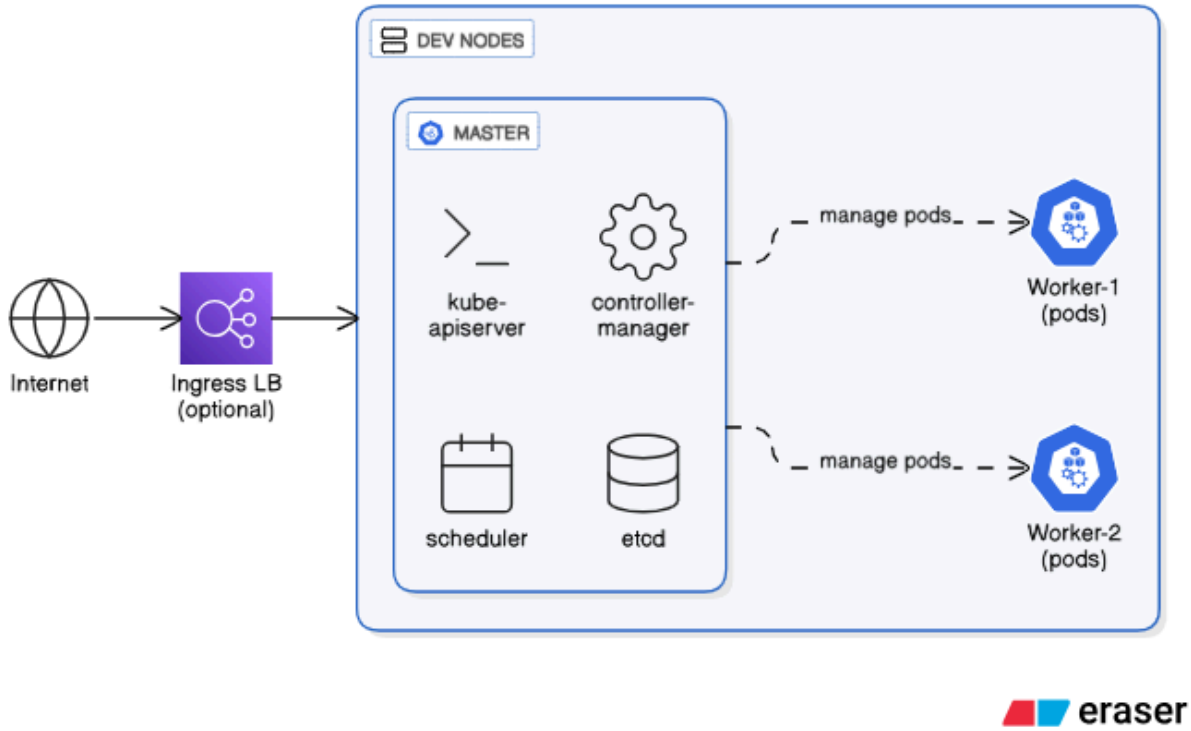
Let's do it.

1) High-level design (short)

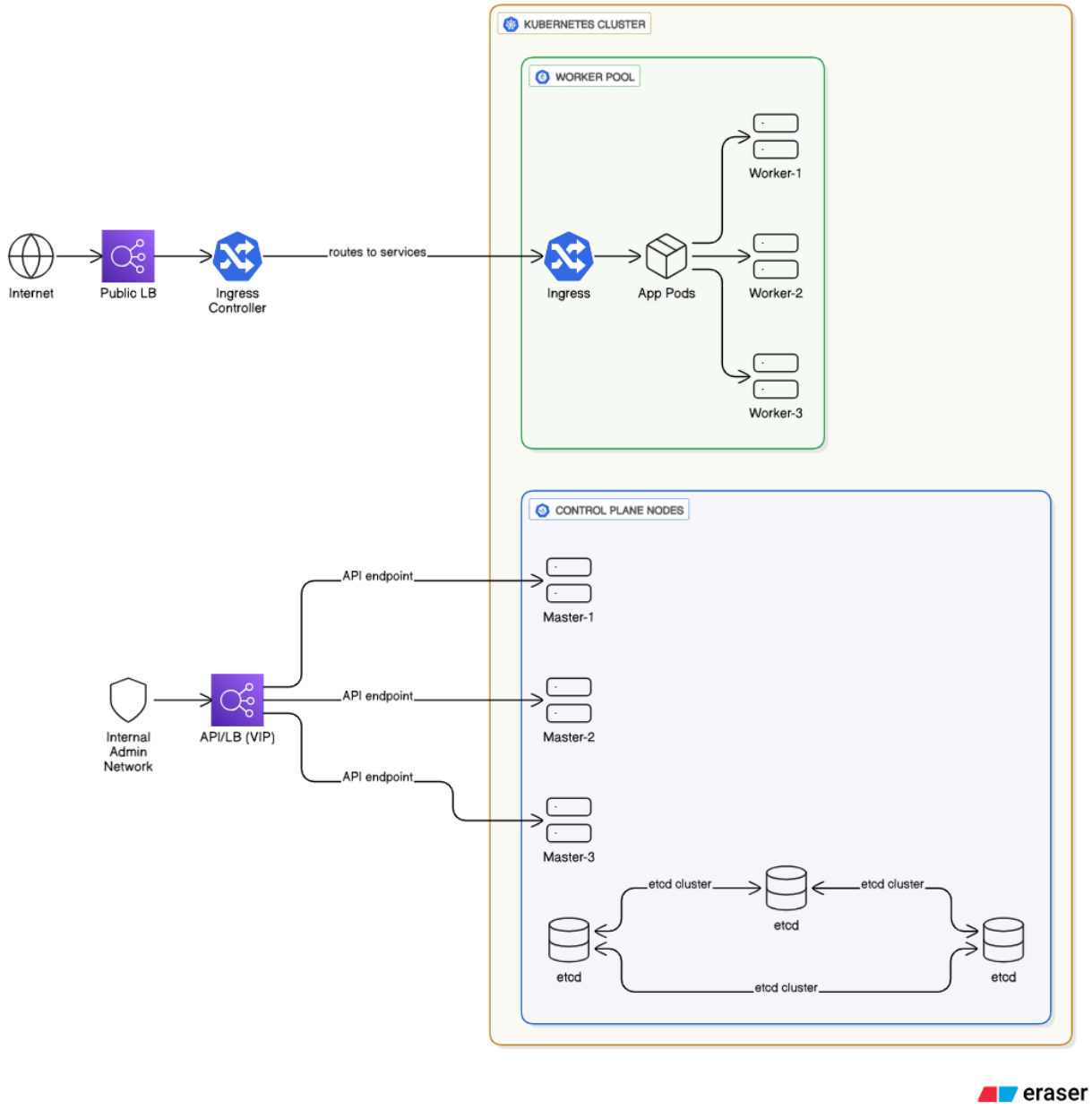
- **Dev cluster**
    - Purpose: developer sandbox, feature testing.
    - Control plane: 1 master (not HA).
    - Workers: 2 small nodes.
    - LB: not required (use NodePort or port-forward during dev) or optional single small LB.
- **Test cluster**
    - Purpose: integration / staging. Mirror prod features.
    - Control plane: 3 masters (HA) or 1 master if constrained (prefer 3).
    - Workers: 3 medium nodes.
    - External Load Balancer in front of control plane (for kube-apiserver) + ingress load balancer for apps.

- **Prod cluster**
    - Purpose: production. Highly available and secure.
    - Control plane: 3 (or 5) masters (HA).
    - Workers: node pools (web, app, batch) — e.g., 3-6 nodes per pool depending on load.
    - External Load Balancers:
        - Fronting kube-apiserver (controlPlaneEndpoint) for HA kubeadm.
        - Fronting Ingress controller (public traffic).
    - etcd: stacked on control-plane or external etcd cluster (external etcd recommended for larger scale).

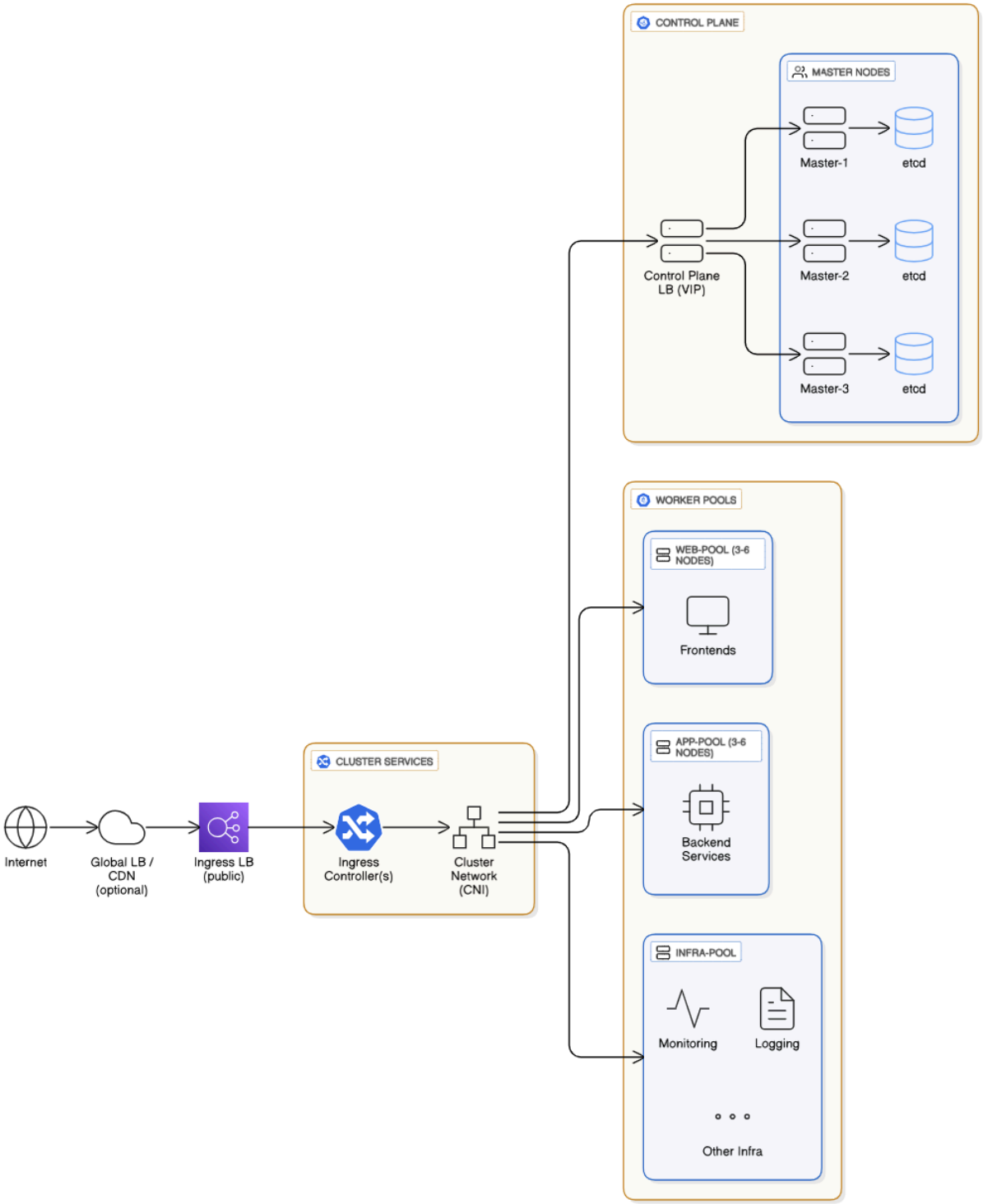
Dev cluster (single master)



Test cluster (HA)



Prod cluster (recommended HA + node pools)


CONTROL PLANE
MASTER NODES
Master-1
etcd
Control Plane LB (VIP)
Master-2
etcd
Master-3
etcd
WORKER POOLS
WEB-POOL (3-6 NODES)
Frontends
APP-POOL (3-6 NODES)
Backend Services
INFRA-POOL
Monitoring
Logging
Other Infra
CLUSTER SERVICES
Internet
Global LB / CDN (optional)
Ingress LB (public)
Ingress Controller(s)
Cluster Network (CNI)

eraser

B. kubeadm init (dev: single master)

On master node (or driven via Ansible `run_once: true` task) run:

```
sudo kubeadm init \
  --pod-network-cidr=192.168.0.0/16 \
  --kubernetes-version=v1.26.0 \
  --apiserver-advertise-address=<MASTER_IP>
```

Then copy kubeconfig for user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Install CNI (example Calico):

```
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

Join workers using the `kubeadm join ...` command printed by `kubeadm init`.

C. kubeadm HA control plane (prod/test) — example kubeadm-config.yaml

You need an **external load balancer** (virtual IP) `CONTROL_PLANE_LB:6443` that routes to all master nodes.

Sample cluster config (save `kubeadm-config.yaml` on the first master):

```
apiVersion: kubeadm.k8s.io/v1beta3
kind: ClusterConfiguration
kubernetesVersion: "v1.26.0"
controlPlaneEndpoint: "k8s-api.example.com:6443"   # external LB
public/cluster IP
networking:
  podSubnet: "192.168.0.0/16"
```

```
---
apiVersion: kubeadm.k8s.io/v1beta3
kind: InitConfiguration
nodeRegistration:
  kubeletExtraArgs:
    node-labels: "node-role.kubernetes.io/control-plane="
    authorization-mode: AlwaysAllow
```

Run on first master:

```
sudo kubeadm init --config kubeadm-config.yaml --upload-certs
```

This prints:

- `kubeadm join <api-endpoint> --token ... --discovery-token-ca-cert-hash sha256:...` for worker nodes
- `kubeadm join <api-endpoint> --control-plane --token ... --discovery-token-ca-cert-hash sha256:... --certificate-key ...` for control-plane nodes

Use the `--certificate-key` value to join additional control-plane nodes:

```
sudo kubeadm join k8s-api.example.com:6443 --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane --certificate-key <certkey>
```

> Production note: keep `--certificate-key` secure; it expires.

---

4) Deploy a simple webapp + Service + Ingress (concrete manifests)

Use a tiny nginx app with a custom index page.

webapp deployment (nginx) — `webapp-deployment.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: simple-webapp
  labels:
    app: simple-webapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: simple-webapp
  template:
    metadata:
      labels:
        app: simple-webapp
    spec:
      containers:
        - name: web
          image: nginx:stable
          ports:
            - containerPort: 80
          volumeMounts:
            - name: index
              mountPath: /usr/share/nginx/html/index.html
              subPath: index.html
      volumes:
        - name: index
          configMap:
            name: simple-index
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: simple-index
data:
```

```
  index.html: |
    <html>
      <head><title>Simple Webapp</title></head>
      <body>
        <h1>Simple Webapp</h1>
        <p>Deployed in the Kubernetes cluster. Host:
<strong>demo.example.com</strong></p>
      </body>
    </html>
```

Service (ClusterIP) — `webapp-service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: simple-webapp
spec:
  selector:
    app: simple-webapp
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
```

Ingress (using nginx-ingress) — `webapp-ingress.yaml`

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-webapp-ingress
  annotations:
    kubernetes.io/ingress.class: "nginx"   # depends on your ingress
controller
    cert-manager.io/cluster-issuer: "letsencrypt-prod"  # optional for
TLS
spec:
  rules:
```

```
    - host: demo.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: simple-webapp
                port:
                  number: 80
  # optional TLS block:
  # tls:
  #   - hosts:
  #       - demo.example.com
  #     secretName: demo-tls
```

Install the ingress controller (NGINX via Helm)

1. Add repo:

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
```

2. Install into `ingress-nginx` namespace:

```
kubectl create namespace ingress-nginx
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --set controller.publishService.enabled=true
```

> If running on bare-metal, you may need MetalLB for a LoadBalancer service type.
> In cloud, the LoadBalancer type will provision cloud LB.

Expose ingress externally

- In cloud: ingress-nginx Service type `LoadBalancer` gets external IP. Point DNS `demo.example.com` to that IP.
- On-prem / bare-metal: use MetalLB, or set ingress controller as NodePort and external LB to map host ports 80/443 to nodes.

---

5) Example full sequence to deploy the webapp

1. Install nginx-ingress controller (see previous helm commands).
2. Apply app manifests:

```
kubectl apply -f webapp-deployment.yaml
kubectl apply -f webapp-service.yaml
kubectl apply -f webapp-ingress.yaml
```

3. Ensure DNS `demo.example.com` points to the ingress controller external IP.
4. Hit `http://demo.example.com/` in browser.

---

6) Extra operational considerations (short bullets)

- **CNI**: pick Calico/Cilium. They affect network policy and pod networking.
- **Storage**: Use dynamic PVs (EBS/Cinder/Managed storage) for stateful components. For the above, webapp is stateless.

- **Backups**: etcd backups (cron + store in object storage). For Mongo/Elastic/Graylog use native backups/snapshots.
- **Monitoring**: Prometheus + kube-state-metrics + Grafana. Install via Helm. Node exporter on hosts.
- **Logging**: EFK/ELK or cluster logging via Fluentd/Fluentbit to external Elasticsearch.
- **Ingress TLS**: use cert-manager to automate Let's Encrypt certs.
- **Security**: PodSecurity admission, RBAC least privileges, network policies, image scanning.
- **Autoscaling**: HPA for pods, cluster-autoscaler for node pools.
- **CI/CD**: use GitOps (ArgoCD/Flux) to manage deployments declaratively.